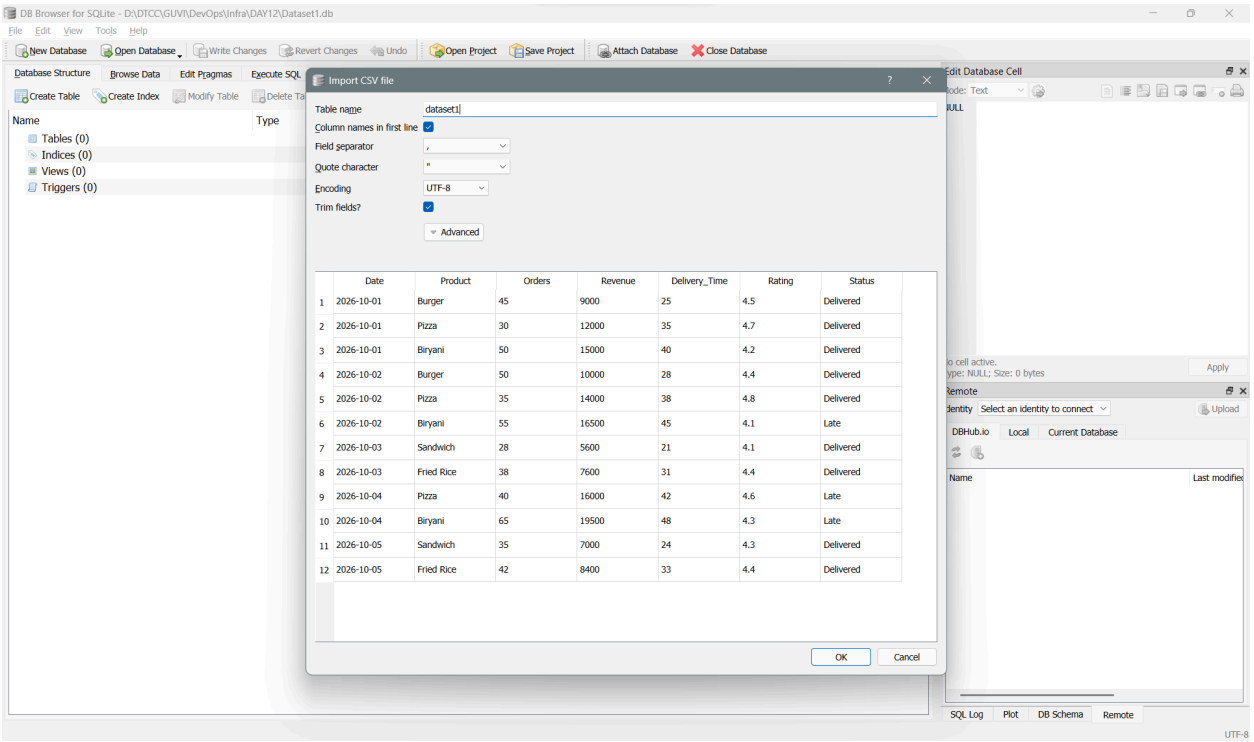
Database Creation, Table Design, and SQL Query Execution Using Dataset-1 (Food Delivery Dataset)

Dataset1.csv

```
Date,Product,Orders,Revenue,Delivery_Time,Rating,Status
2026-10-01,Burger,45,9000,25,4.5,Delivered
2026-10-01,Pizza,30,12000,35,4.7,Delivered
2026-10-01,Biryani,50,15000,40,4.2,Delivered
2026-10-02,Burger,50,10000,28,4.4,Delivered
2026-10-02,Pizza,35,14000,38,4.8,Delivered
2026-10-02,Biryani,55,16500,45,4.1,Late
2026-10-03,Sandwich,28,5600,21,4.1,Delivered
2026-10-03,Fried Rice,38,7600,31,4.4,Delivered
2026-10-04,Pizza,40,16000,42,4.6,Late
2026-10-04,Biryani,65,19500,48,4.3,Late
2026-10-05,Sandwich,35,7000,24,4.3,Delivered
2026-10-05,Fried Rice,42,8400,33,4.4,Delivered
```

Importing in SQLLite



Query 1 — Display all records (SELECT + ORDER BY)

```
SELECT *
FROM dataset1
ORDER BY Date, Product;
```

Output:

	Date	Product	Orders	Revenue	Delivery_Time	Rating	Status
1	2026-10-01	Biryani	50	15000	40	4.2	Delivered
2	2026-10-01	Burger	45	9000	25	4.5	Delivered
3	2026-10-01	Pizza	30	12000	35	4.7	Delivered
4	2026-10-02	Biryani	55	16500	45	4.1	Late
5	2026-10-02	Burger	50	10000	28	4.4	Delivered
6	2026-10-02	Pizza	35	14000	38	4.8	Delivered
7	2026-10-03	Fried Rice	38	7600	31	4.4	Delivered
8	2026-10-03	Sandwich	28	5600	21	4.1	Delivered
9	2026-10-04	Biryani	65	19500	48	4.3	Late
10	2026-10-04	Pizza	40	16000	42	4.6	Late
11	2026-10-05	Fried Rice	42	8400	33	4.4	Delivered
12	2026-10-05	Sandwich	35	7000	24	4.3	Delivered

Explanation: Displays all rows and columns in dataset1. ORDER BY sorts the results by date and then product name so the records are easier to review.

Query 2 — Find late deliveries (SELECT + WHERE + ORDER BY)

```
SELECT Date, Product, Orders, Delivery_Time, Status
FROM dataset1
WHERE Status = 'Late'
ORDER BY Delivery Time DESC;
```

Output:

	Date	Product	Orders	Delivery_Time	Status
1	2026-10-04	Biryani	65	48	Late
2	2026-10-02	Biryani	55	45	Late
3	2026-10-04	Pizza	40	42	Late

Explanation: WHERE filters the data to records marked Late. ORDER BY Delivery_Time DESC places the longest delivery times first.

Query 3 — Overall business summary (aggregate functions)

```sql
SELECT
    COUNT(*) AS Total_Records,
    SUM(Orders) AS Total_Orders,
    SUM(Revenue) AS Total_Revenue,
    ROUND(AVG(Delivery_Time), 2) AS Avg_Delivery_Minutes,
    ROUND(AVG(Rating), 2) AS Avg_Rating
FROM dataset1;
```

Output:

	Total_Records	Total_Orders	Total_Revenue	Avg_Delivery_Minutes	Avg_Rating
1	12	513	140600	34.17	4.4

Explanation: COUNT, SUM, and AVG calculate the number of records, total orders, total revenue, average delivery time, and average rating across the complete dataset.

Query 4 — Performance by product (GROUP BY + aggregate functions)

```sql
SELECT
    Product,
    SUM(Orders) AS Total_Orders,
    SUM(Revenue) AS Total_Revenue,
    ROUND(AVG(Delivery_Time), 2) AS Avg_Delivery_Minutes,
    ROUND(AVG(Rating), 2) AS Avg_Rating
FROM dataset1
GROUP BY Product
ORDER BY Total Revenue DESC;
```

Output:

	Product	Total_Orders	Total_Revenue	Avg_Delivery_Minutes	Avg_Rating
1	Biryani	170	51000	44.33	4.2
2	Pizza	105	42000	38.33	4.7
3	Burger	95	19000	26.5	4.45
4	Fried Rice	80	16000	32.0	4.4
5	Sandwich	63	12600	22.5	4.2

Explanation: GROUP BY combines records for each product. Aggregate functions calculate each product's total orders and revenue, plus its average delivery time and rating. The highest-revenue product appears first.

Query 5 — Products with revenue above 20,000 (GROUP BY + HAVING)

```sql
SELECT
    Product,
    SUM(Revenue) AS Total_Revenue
FROM dataset1
GROUP BY Product
HAVING SUM(Revenue) > 20000
ORDER BY Total_Revenue DESC;
```

Output:

	Product	Total_Revenue
1	Biryani	51000
2	Pizza	42000

Explanation: HAVING filters grouped results after revenue is summed for each product. Only products with total revenue greater than 20,000 are returned.

Query 6 — JOIN each record with its product revenue total

```sql
SELECT
    d.Date,
    d.Product,
    d.Revenue,
    p.Product_Total_Revenue
FROM dataset1 AS d
JOIN (
    SELECT Product, SUM(Revenue) AS Product_Total_Revenue
    FROM dataset1
    GROUP BY Product
) AS p
    ON d.Product = p.Product
ORDER BY d.Date, d.Product;
```

Output:

	Date	Product	Revenue	Product_Total_Revenue
1	2026-10-01	Biryani	15000	51000
2	2026-10-01	Burger	9000	19000
3	2026-10-01	Pizza	12000	42000
4	2026-10-02	Biryani	16500	51000
5	2026-10-02	Burger	10000	19000
6	2026-10-02	Pizza	14000	42000
7	2026-10-03	Fried Rice	7600	16000
8	2026-10-03	Sandwich	5600	12600
9	2026-10-04	Biryani	19500	51000
10	2026-10-04	Pizza	16000	42000
11	2026-10-05	Fried Rice	8400	16000
12	2026-10-05	Sandwich	7000	12600

Explanation: This INNER JOIN matches each original row (alias d) with a grouped subquery (alias p) that calculates total revenue by product. It demonstrates JOIN without needing to invent a second source table.

Query 7 — Compare delivered and late records (GROUP BY + aggregate functions)

```sql
SELECT
    Status,
    COUNT(*) AS Record_Count,
    SUM(Orders) AS Total_Orders,
    SUM(Revenue) AS Total_Revenue,
    ROUND(AVG(Delivery_Time), 2) AS Avg_Delivery_Minutes,
    ROUND(AVG(Rating), 2) AS Avg_Rating
FROM dataset1
GROUP BY Status
ORDER BY Total_Revenue DESC;
```

Output:

	Status	Record_Count	Total_Orders	Total_Revenue	Avg_Delivery_Minutes	Avg_Rating
1	Delivered	9	353	88600	30.56	4.42
2	Late	3	160	52000	45.0	4.33

Explanation: GROUP BY compares the records by delivery status. The output shows the record count, order total, revenue total, average delivery time, and average rating for each status.